

Conceptos de Sistemas Operativos (IC)
Introducción a los Sistemas Operativos (LI, LS, APU, ATIC)

Trabajo Práctico N° 2

Objetivo

Comprender conceptos básicos de los sistemas operativos, los procesos y su planificación para la administración de la CPU. Introducir y comprender la administración de memoria, considerando las diferentes posibilidades acordes a las características del hardware, distinguiendo el apoyo de este así como las actividades a realizar para la eficiente administración del recurso por parte del Kernel.

Se recomienda, para la autocorrección de los ejercicios, la utilización del simulador de planificación que se publicará en la plataforma.

Temas incluidos

Definición de SO. Componentes de un SO. Apoyo del Hardware: Modos de ejecución, interrupciones. Llamadas al sistema. Programa y Proceso. Planificación de CPU. Colas de planificación. Context Switch. Creación de procesos. Espacio de direcciones. Direcciones lógicas y físicas. Particiones fijas y dinámicas. Paginación y Segmentación. Fragmentación. MMU. Algoritmos de administración.

1. Responda en forma sintética sobre los siguientes conceptos:
 - a. ¿Qué es un Sistema Operativo?
 - b. Enumere qué componentes/aspectos del Hardware son necesarios para cumplir los objetivos de un Sistema Operativo.
 - c. Enumere componentes de un Sistema Operativo
 - d. ¿Que es una llamada al sistema (*system call*)? ¿Cómo es posible implementarlas?
 - e. Defina y diferencie Programa y Proceso.
 - f. ¿Cuál es la información mínima que el Kernel debe tener sobre un proceso? ¿En qué estructura de datos asociada almacena dicha información?
 - g. ¿Qué objetivos persiguen los algoritmos de planificación (scheduling).
 - h. ¿Qué significa que un algoritmo de scheduling sea apropiativo o no apropiativo (Preemptive o Non-Preemptive)?
 - i. ¿Qué tareas realizan los siguientes módulos de planificación?:
 - i. Short Term Scheduler
 - ii. Long Term Scheduler
 - iii. Medium Term Scheduler
 - j. ¿Qué tareas realiza el Dispatcher? ¿Y el Loader?
 - k. ¿Qué significa que un proceso sea "CPU Bound" y "I/O Bound"?
 - l. ¿Cuáles son los estados posibles por los que puede atravesar un proceso? ¿Qué representa que un proceso se encuentre en los estados

enumerados? Utilizando un diagrama explique las transiciones entre los estados.

- m. ¿Cuáles de los schedulers mencionados anteriormente se encargan de las transiciones entre los estados enumerados?
 - n. Defina Tiempo de retorno (**TR**) y Tiempo de espera (**TE**) para un proceso.
 - o. Defina Tiempo Promedio de Retorno (**TPR**) y Tiempo promedio de espera (**TPE**) para un lote de procesos.
 - p. Defina tiempo de respuesta.
2. Para los siguientes algoritmos de scheduling:
 - FCFS (First Come First Served)
 - SJF (Shortest Job First)
 - Round Robin
 - Prioridades
 - a. Explique su funcionamiento mediante un ejemplo.
 - b. ¿Alguno de ellos cuentan con parámetros para su funcionamiento? Identifique y enunciarlos
 - c. Cual es el más adecuado según los tipos de procesos y/o SO.
 - d. Cite ventajas y desventajas de su uso.
 3. Dado el siguiente lote de procesos en el que todos arriban al sistema en el instante 0 (cero):

Job	Unidades de CPU
1	7
2	15
3	12
4	4
5	9

- a. Realice los diagramas de Gantt según los siguientes algoritmos scheduling:
 - i. FCFS (First Come, First Served)
 - ii. SJF (Shortest Job First)
 - iii. Round Robin con quantum = 4
- b. Para cada algoritmo calcule el TR y TE para cada job así como el TPR y el TPE.
- c. En base a los tiempos calculados compare los diferentes algoritmos.
4. Dado el siguiente lote procesos:

Job	Llegada	Unidades de CPU
1	0	4
2	2	6
3	3	4

4	6	5
5	8	2

- Realice los diagramas de Gantt según los siguientes algoritmos de scheduling:
 - FCFS (First Come, First Served)
 - SJF (Shortest Job First)
 - Round Robin con quantum = 1
 - Round Robin con quantum = 6
 - Para cada algoritmo calcule el TR y TE para cada job así como el TPR y el TPE.
 - En base a los tiempos calculados compare los diferentes algoritmos.
 - En el algoritmo Round Robin, qué conclusión se puede sacar con respecto al valor del quantum.
 - ¿Para el algoritmo Round Robin, en qué casos utilizará un valor de quantum alto y qué ventajas y desventajas obtendría?
5. Una variante al algoritmo SJF es el algoritmo SJF apropiativo o SRTF (Shortest Remaining Time First):
- Realice el diagrama de Gantt para este algoritmo según el lote de trabajos del ejercicio 5.
 - ¿Nota alguna ventaja frente a otros algoritmos?
6. Suponga que se agregan las siguientes prioridades al lote de procesos del ejercicio 5, donde un menor número indica mayor prioridad:

Job	Prioridad
1	3
2	4
3	2
4	1
5	2

- Realice el diagrama de Gantt correspondiente al algoritmo de planificación por prioridades según las variantes:
 - No Apropiativa
 - Apropiativa
 - Calcule el TR y TE para cada job así como el TPR y el TPE.
 - ¿Nota alguna ventaja frente a otros algoritmos? ¿Bajo qué circunstancias lo utilizaría y ante qué situaciones considera que la implementación de prioridades podría no ser de mayor relevancia?
7. Inanición (*Starvation*)
- ¿Qué significa?

- b. ¿Cuál/es de los algoritmos vistos puede provocarla?
- c. ¿Existe alguna técnica que evite la inanición para el/los algoritmos mencionados en el inciso b?

8. Los procesos, durante su ciclo de vida, pueden realizar operaciones de I/O como lecturas o escrituras a disco, cintas, uso de impresoras, etc. El Kernel mantiene para cada dispositivo, que se tiene en el equipo, una cola de procesos que espera por la utilización del mismo (al igual que ocurre con la Cola de Listos y la CPU, ya que la CPU es un dispositivo más).

Cuando un proceso en ejecución realiza una operación de I/O el mismo es expulsado de la CPU y colocado en la cola correspondiente al dispositivo involucrado en la operación.

El Kernel dispone también de un "I/O Scheduling" que administra cada cola de dispositivo a través de algún algoritmo (FCFS, Prioridades, etc.). Si al colocarse un proceso en la cola del dispositivo, la misma se encuentra vacía el mismo será atendido de manera inmediata, caso contrario, deberá esperar a que el Kernel lo seleccione según el algoritmo de scheduling establecido.

Los mecanismos de I/O utilizados hoy en día permiten que la CPU no sea utilizada durante la operación, por lo que el Kernel puede ejecutar otro proceso que se encuentre en espera una vez que el proceso bloqueado por la I/O se coloca en la cola correspondiente.

Cuando el proceso finaliza la operación de I/O el mismo retorna a la cola de listos para competir nuevamente por la utilización de la CPU.

Para los siguientes algoritmos de Scheduling:

- FCFS
- Round Robin con quantum = 2

Y suponiendo que la cola de listos de todos los dispositivos se administra mediante FCFS, realice los diagramas de Gantt según las siguientes situaciones:

- a. Suponga que al lote de procesos del ejercicio 5 se agregan las siguientes operaciones de entrada salida:

Job	I/O (Recurso,Instante,Duración)
1	(R1, 2, 1)
2	(R2, 3, 1) (R2, 5, 2)
4	(R3, 1, 2) (R3, 3, 1)

- b. Suponga que al lote de procesos del ejercicio 5 se agregan las siguientes operaciones de entrada salida:

Job	I/O (Recurso,Instante,Duración)
1	(R1, 2, 3) (R1, 3, 2)
2	(R2, 3, 2)
3	(R2, 2, 3)

4	(R1, 1, 2)
---	------------

9. Algunos algoritmos pueden presentar ciertas desventajas cuando en el sistema se cuenta con procesos ligados a CPU y procesos ligados a entrada salida. Analice las mismas para los siguientes algoritmos:
- Round Robin
 - SRTF (Shortest Remaining Time First)

10. Para equiparar la desventaja planteada en el ejercicio 10, se plantea la siguiente modificación al algoritmo:

Algoritmo VRR (Virtual Round Robin): Este algoritmo funciona igual que el Round Robin, con la diferencia que cuando un proceso regresa de una I/O se coloca en una cola auxiliar. Cuando se tiene que tomar el próximo proceso a ejecutar, los procesos que se encuentran en la cola auxiliar tienen prioridad sobre los otros. Cuando se elige un proceso de la cola auxiliar se le otorga el procesador por tantas unidades de tiempo como le faltó ejecutar en su ráfaga de CPU anterior, esto es, se le otorga la CPU por un tiempo que surge entre la diferencia del quantum original y el tiempo usado en la última ráfaga de CPU.

- Analice el funcionamiento de este algoritmo mediante un ejemplo. Marque en cada instante en que cola se encuentran los procesos.
 - Realice el ejercicio 9)a) nuevamente considerando este algoritmo, con un quantum de 2 unidades
11. Suponga que un Kernel utiliza un algoritmo de VRR para planificar sus procesos. Para ello, el quantum es representado por un contador, que es decrementado en 1 unidad cada vez que ocurre una interrupción de reloj. ¿Bajo este esquema, puede suceder que el quantum de un proceso nunca llegue a 0 (cero)? Justifique su respuesta.
12. El algoritmo SJF (y SRTF) tiene como problema su implementación, dada la dificultad de conocer la duración de la próxima ráfaga de CPU. Es posible realizar una estimación de la próxima, utilizando la media de las ráfagas de CPU para cada proceso. Así, por ejemplo, podemos tener la siguiente fórmula (1):

$$S_{n+1} = \frac{1}{n}T_n + \frac{n-1}{n}S_n$$

Donde:

T_i = duración de la ráfaga de CPU i-ésima del proceso.

S_i = valor estimado para el i-ésimo caso

S_1 = valor estimado para la primera ráfaga de CPU. No es calculado.

- Suponga un proceso cuyas ráfagas de CPU reales tienen como duración: 6, 4, 6, 4, 13, 13, 13. Calcule qué valores se obtendrían como estimación para las ráfagas de CPU del proceso si se utiliza la fórmula 1, con un valor inicial

estimado de $S_1=10$.

La fórmula 1 le da el mismo peso a todos los casos (siempre calcula la media). Es posible reescribirla permitiendo darle un peso mayor a los casos más recientes y menor a casos viejos (o viceversa). Se plantea la siguiente fórmula 2:

$$S_{n+1} = \alpha T_n + (1 - \alpha) S_n$$

Con $0 < \alpha < 1$.

- Analice para qué valores de α se tienen en cuenta los casos más recientes.
- Para la situación planteada en a) calcule qué valores se obtendrían si se utiliza la fórmula 2 con $\alpha = 0, 2$; $\alpha = 0, 5$ y $\alpha = 0,8$.
- Para todas las estimaciones realizadas en a y c ¿Cuál es la que más se asemeja a las ráfagas de CPU reales del proceso?

13. Colas Multinivel

Actualmente los algoritmos de planificación vistos se han ido combinando para formar algoritmos más eficientes. Así surge el algoritmo de Colas Multinivel, donde la cola de procesos listos es dividida en varias colas, teniendo cada una su propio algoritmo de planificación.

- Suponga que se tiene dos tipos de procesos: *Interactivos* y *Batch*. Cada uno de estos procesos se coloca en una cola según su tipo. ¿Qué algoritmo de los vistos utilizará para administrar cada una de estas colas?

A su vez, se utiliza un algoritmo para administrar cada cola que se crea. Así, por ejemplo, el algoritmo podría determinar mediante prioridades sobre qué cola elegir un proceso.

- Para el caso de las dos colas vistas en a: ¿Qué algoritmo utilizaría para planificarlas?

14. Suponga que en un Kernel se utiliza un algoritmo de planificación de colas multinivel. El mismo cuenta con 3 colas de procesos listos, en las que los procesos se encolan en una u otra según su prioridad. Hay 3 prioridades (1, 2, 3), donde un menor número indica mayor prioridad. Se utiliza el algoritmo de prioridades para la administración entre las colas.

Se tiene el siguiente lote de procesos a ser procesados con sus respectivas operaciones de I/O:

Job	Llegada	CPU	I/O (rec,ins,dur)	Prioridad
1	0	9	(R1, 4, 2) (R2, 6, 3) (R1, 8, 3)	1
2	1	5	(R3, 3, 2) (R3, 4, 2)	2
3	2	5	(R1, 4, 1)	3
4	3	7	(R2, 1, 2) (R2, 5, 3)	2
5	5	5	(R1, 2, 3) (R3, 4, 3)	1

Suponiendo que las colas de cada recurso (dispositivo de entrada/salida) se

administran a través de FCFS y que cada cola de procesos listos se administra por medio de un algoritmo RR con un quantum de 3 unidades, realice un diagrama de Gantt:

- a. Asumiendo que NO hay apropiación entre los procesos.
- b. Asumiendo que hay apropiación entre los procesos.

15. En el esquema de Colas Multinivel, cuando se utiliza un algoritmo de prioridades para administrar las diferentes colas los procesos pueden sufrir starvation.

La técnica de envejecimiento se puede aplicar a este esquema, haciendo que un proceso cambie de una cola de menor prioridad a una de mayor prioridad, después de cierto periodo de tiempo que el mismo se encuentra esperando en su cola. Luego de llegar a una cola en la que el proceso llega a ser atendido, el mismo retorna a su cola original.

Por ejemplo: Un proceso con prioridad 3 está en cola su cola correspondiente. Luego de X unidades de tiempo, el proceso se mueve a la cola de prioridad 2. Si en esta cola es atendido, retorna a su cola original, en caso contrario luego de sucederse otras X unidades de tiempo el proceso se mueve a la cola de prioridad 1. Esta última acción se repite hasta que el proceso obtiene la CPU, situación que hace que el mismo vuelva a su cola original.

Para los casos a y b del ejercicio 15 realice el diagrama de Gantt considerando además que se tiene un envejecimiento de 4 unidades.

16. La situación planteada en el ejercicio 16, donde un proceso puede cambiar de una cola a otra, se la conoce como Colas Multinivel con Realimentación.

Suponga que se quiere implementar un algoritmo de planificación que tenga en cuenta el tiempo de ejecución consumido por el proceso, penalizando a los que más tiempo de ejecución tienen. (Similar a la tarea del algoritmo SJF que tiene en cuenta el tiempo de ejecución que resta).

Utilizando los conceptos vistos de Colas Multinivel con Realimentación indique qué colas implementaría, qué algoritmo usaría para cada una de ellas así como para la administración de las colas entre sí.

Tenga en cuenta que los procesos no deben sufrir inanición.

17. A cuáles de los siguientes tipos de trabajos:

- cortos acotados por CPU
- cortos acotados por E/S
- largos acotados por CPU
- largos acotados por E/S

benefician las siguientes estrategias de administración:

- a. prioridad determinada estáticamente con el método del más corto primero (SJF).
- b. prioridad dinámica inversamente proporcional al tiempo transcurrido desde la última operación de E/S.

18. Analizar y explicar por qué si el quantum en Round-Robin se incrementa sin límite, el algoritmo de planificación se aproxima a FIFO.

19. Dado los siguientes programas en un pseudo-código que simula la utilización de llamadas al sistema para crear procesos en Unix/Linux, indicar qué mensajes se imprimirán en pantalla (sin importar el orden en que saldrían):

a. Caso 1

```
printf("hola")
x = fork()
if x < 1 {
    execv("ls")
    printf("mundo")
    exit(0)
}
exit(0)
```

b. Caso 2

```
printf("hola")
x = fork()
if x < 1 {
    execv("ps")
    printf("mundo")
    exit(0)
}
execv("ls")
printf("fin")
exit(0)
```

c. Caso 3

```
printf("Anda a rendir el Primer Parcial de
Promo!")
newpid = fork()
if newpid == 0 {
    printf("Estoy comenzando el Examen")
    execv("ps")
    printf("Termine el Examen")
}
printf("¿Como te fue?")
exit(0)
printf("Ahora anda a descansar")
```

20. Dado el siguiente programa en C, analice su objetivo sin necesidad de ejecutarlo. (Investigue el funcionamiento del iterador *for* en C para poder responder:

<https://ccia.ugr.es/~jfv/ed1/c/cdrom/cap4/cap43.htm>)

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main ( void ) {
    int c ;
    pid_t pid ;
    printf( " Comienzo . : \ n " ) ;
    for ( c = 0 ; c < 3 ; c++ ) {
```

```
        pid = fork ( ) ;  
    }  
    printf ( " Proceso \n " ) ;  
    return 0 ;  
}
```

- ¿Cuántas líneas con la palabra "Proceso" aparecen al final de la ejecución de este programa?
- ¿El número de líneas es el número de procesos que han estado en ejecución?. Ejecute el programa y compruebe si su respuesta es correcta. Modifique el valor del bucle for y compruebe los nuevos resultados.

21. Modifiquemos el programa anterior. Ahora, además de un mensaje, vamos a añadir una variable y antes de finalizar el programa vamos a mostrar el valor de la misma:

```
#include <stdio.h>  
#include <sys/types.h>  
#include <unistd.h>  
int main ( void ) {  
    int c ;  
    pid_t pid ;  
    int p = 0;  
    printf( " Comienzo . : \n " ) ;  
    for ( c = 0 ; c < 3 ; c++ ) {  
        pid = fork ( ) ;  
    }  
    p++;  
    printf ( " Proceso %d\n ",p ) ;  
    return 0 ;  
}
```

- ¿Qué valores se imprimirán en la consola?
- ¿Todas las líneas tendrán el mismo valor o algunas líneas tendrán valores distintos?
- ¿Cuál es el valor (o valores) que aparece?. Compile y ejecute el programa y compruebe si su respuesta es correcta.
- Realice modificaciones al programa, por ejemplo: modifique el valor del bucle for, cambie el lugar dónde se incrementa la variable p, y compruebe los nuevos resultados.

22. ¿Qué es la MMU y qué funciones cumple?

23. ¿Que representa el espacio de direcciones de un proceso?

24. Explique y relacione los siguientes conceptos:

Dirección Lógica o Virtual
Dirección Física

25. En la técnica de Particiones Múltiples, la memoria es dividida en varias particiones y

los espacios de direcciones de los procesos son ubicados en estas, siempre que el tamaño del mismo sea menor o igual que el tamaño de la partición.

Al trabajar con particiones se pueden considerar 2 métodos (independientes entre sí):

Particiones Fijas

Particiones Dinámicas

- Explique cómo trabajan estos 2 métodos. Cite diferencias, ventajas y desventajas.
- ¿Qué información debe disponer el Kernel para poder administrar la memoria principal con estos métodos?
- Realice un gráfico indicando cómo se realiza la transformación de direcciones lógicas a direcciones físicas.

26. Al trabajar con particiones fijas, los tamaños de las mismas se pueden considerar:

- Particiones de igual tamaño.
- Particiones de diferente tamaño.

Cite ventajas y desventajas entre las alternativas.

27. Fragmentación. Ambos métodos de particiones presentan el problema de la fragmentación:

Fragmentación Interna (Para el caso de Particiones Fijas)

Fragmentación Externa (Para el caso de Particiones Dinámicas)

- Explique a qué hacen referencia estos 2 problemas
- El problema de la Fragmentación Externa es posible de subsanar. Explique una posible técnica que permita al Kernel mitigar este problema.

28. Analice y describa cómo funcionan las siguientes técnicas de asignación de particiones: Best Fit, Worst Fit, First Fit y Next Fit. Tenga en cuenta que información debe mantener el Kernel. Indique, para los métodos de particiones dinámicas y fijas, con cuál técnica se obtienen mejores resultados y justifique.

29. Segmentación

- Explique cómo trabaja este método de asignación de memoria.
- ¿Qué estructuras son necesarias en el Kernel para poder ejecutarse sobre un Hardware que utiliza segmentación?
- Explique, utilizando gráficos, cómo son resueltas las direcciones lógicas a físicas.
- En este esquema: ¿se puede producir fragmentación (interna y/o externa)?
- De las técnicas enumeradas en el ejercicio 29 ¿Cuál se ajusta mejor para la asignación de memoria principal?

30. Dado un esquema de segmentación donde cada dirección hace referencia a 1 byte y la siguiente tabla de segmentos de un proceso, traduzca, de corresponder, las

direcciones lógicas indicadas a direcciones físicas. La Dirección lógica está representada por: **segmento:desplazamiento**

Segmento	Dir. Base	Tamaño
0	102	12500
1	28699	24300
2	68010	15855
3	80001	400

- | | |
|----------------|----------------|
| i. 0000:9001 | iv. 0001:18976 |
| ii. 0001:24301 | v. 0003:0 |
| iii. 0002:5678 | |

31. Paginación

- Explique cómo trabaja este método de asignación de memoria.
- ¿Qué estructuras son necesarias en el Kernel para poder ejecutarse sobre un Hardware que utiliza paginación?
- Explique, utilizando gráficos, cómo son resueltas las direcciones lógicas en físicas.
- En este esquema: ¿se puede producir fragmentación (interna y/o externa)?

32. Suponga un sistema donde la memoria es administrada mediante la técnica de paginación, y donde:

El tamaño de la página es de 512 bytes

Cada dirección de memoria referencia 1 byte.

Se tiene una memoria principal de 10240 bytes (10 KiB) y los marcos se enumeran desde 0 y comienzan en la dirección física 0.

Suponga además un proceso P1 con un tamaño lógico de 2000 bytes y con la siguiente tabla de páginas:

Página	Marco
0	3
1	5
2	2
3	6

- Realice los gráficos necesarios (de la memoria principal, del espacio de direcciones del proceso, de tabla de páginas y de la tabla de marcos) en el que refleje el estado descripto.
- Indicar si las siguientes direcciones lógicas corresponden al espacio lógico del proceso P1 y en caso afirmativo indicar la dirección física a la que corresponden:

- | | |
|-----------|---------|
| i. 35 | iv. 0 |
| ii. 512 | v. 1325 |
| iii. 2051 | vi. 602 |

- c. Indicar, en caso de ser posible, las direcciones lógicas del proceso P1 que se corresponden a las siguientes direcciones físicas:

- | | |
|----------|----------|
| i. 509 | iv. 3215 |
| ii. 1500 | v. 1024 |
| iii. 0 | vi. 2000 |

33. Dado un esquema donde cada dirección hace referencia a 1 byte, con páginas de 2 KiB (KibiBytes), donde el frame 0 se encuentra en la dirección física 0. Con las siguientes siguientes primeras entradas de la tabla de páginas de un proceso,

Página	Marco
0	16
1	13
2	9
3	2
4	0

traduzca las direcciones lógicas indicadas a direcciones físicas:

- | | | |
|----------|-----------|---------|
| i. 5120 | iii. 1578 | v. 8191 |
| ii. 3242 | iv. 2048 | |

34. Tamaño de la Página. Compare las siguientes situaciones con respecto al tamaño de página y su impacto, indicando ventajas y desventajas:

- Un tamaño de página pequeño.
- Un tamaño de página grande.

35. Existen varios enfoques para administrar las tablas de páginas:

- ☐ Tablas de páginas de 1 nivel.
- ☐ Tablas de páginas de 2 niveles o más
- ☐ Tablas de páginas invertidas.

Explique brevemente cómo trabajan estos mecanismos. Tenga en cuenta analizar cómo se resuelven las direcciones y qué ventajas/desventajas presentan una sobre otra.

36. Dado un esquema de paginación, donde cada dirección hace referencia a 1 byte, se tiene un tamaño de dirección de 32 bits y un tamaño de página de 2048 bytes. Suponga además un proceso P1 que necesita 51358 bytes para su código y 68131 bytes para sus datos.

- De los 32 bits de las direcciones ¿Cuántos se utilizan para identificar página? ¿Cuántos para desplazamiento?
- ¿Cuál sería el tamaño lógico máximo de un proceso?
- ¿Cuántas páginas máximo puede tener el proceso P1?
- Si se contaran con 4GiB (Gibibytes) de RAM, ¿cuántos marcos habría disponibles para utilizar?
- ¿Cuántas páginas necesita P1 para almacenar su código?
- ¿Cuántas páginas necesita P1 para almacenar sus datos?

- g. ¿Cuántos bytes habría de fragmentación interna entre lo necesario para almacenar su código y sus datos? Ayuda: Recuerde que en una misma página no pueden convivir código y datos.
- h. Asumiendo que el hardware utiliza tabla de páginas de un (1) nivel, que la dirección se interpreta como 20 bits para identificar página y 12 bits para desplazamiento, y que cada Entrada de Tabla de Páginas (PTE) requiere de 1 Kib. ¿Cuál será el tamaño mínimo que la tabla de páginas del proceso P1 ocupará en RAM?
- i. Asumiendo que el hardware utiliza tabla de páginas de dos (2) niveles, que la dirección se interpreta como 10 bits para identificar primer nivel, 10 bits para identificar el segundo nivel, 12 bits para desplazamiento y que cada Entrada de Tabla de Páginas (PTE) requiere de 1 Kib. ¿Cuál será el tamaño mínimo que la tabla de páginas del proceso P1 ocupará en RAM?
- j. Dado los resultados obtenidos en los dos incisos anteriores (h, i) ¿Qué conclusiones se pueden obtener del uso de los 2 modelos de tablas de página?
37. Cite similitudes y diferencias entre las 4 técnicas vistas: Particiones Fijas, Particiones Dinámicas, Segmentación y Paginación.
38. Dada la técnica de administración de memoria por medio de segmentación paginada:
- Explique cómo funciona.
 - Cite ventajas y desventajas respecto a las técnicas antes vistas.
 - Teniéndose disponibles las siguientes tablas:

Tabla de Segmentos		Tabla de Páginas		
Núm. Seg.	Dir. base	Nro. Segmento	Nro. Página	Direc. Base
1	500	1	1	40
			2	80
			3	60
2	1500	2	1	20
			2	25
			3	0
3	5000	3	1	120
			2	150

Indicar las direcciones físicas correspondientes a las siguientes direcciones lógicas (segmento,página,desplazamiento):

- | | |
|--------------|---------------|
| i. (2,1,1) | iii. (3,1,10) |
| ii. (1,3,15) | iv. (2,3,5) |

Conceptos de Sistemas Operativos (IC)

Introducción a los Sistemas Operativos (LI, LS, APU, ATIC)

Trabajo Práctico N° 3

Objetivo

Introducir a los estudiantes en el armado y ejecución de scripts en bash, Shell Scripting.

Temas incluidos

Bash. Script. Sintaxis. GNU/Linux

1. ¿Qué es el Shell Scripting? ¿A qué tipos de tareas están orientados los script? ¿Los scripts deben compilarse? ¿Por qué?
2. Investigar la funcionalidad de los comandos echo y read.
 - a. ¿Cómo se indican los comentarios dentro de un script?
 - b. ¿Cómo se declaran y se hace referencia a variables dentro de un script?
3. Crear dentro del directorio personal del usuario logueado un directorio llamado *practica-shell-script* y dentro de él un archivo llamado *mostrar.sh* cuyo contenido sea el siguiente:

```
#!/bin/bash
# Comentarios acerca de lo que hace el script
# Siempre comento mis scripts, si no lo hago hoy,
# mañana ya no me acuerdo de lo que quise hacer
echo "Introduzca su nombre y apellido:"
read nombre apellido
echo "Fecha y hora actual:"
date
echo "Su apellido y nombre es:"
echo "$apellido $nombre"
echo "Su usuario es: `whoami`"
echo "Su directorio actual es:"
```

- a. Asignar al archivo creado los permisos necesarios de manera que pueda ejecutarlo
- b. Ejecutar el archivo creado de la siguiente manera: `./mostrar`
- c. ¿Qué resultado visualiza?
- d. Las backquotes (```) entre el comando `whoami` ilustran el uso de la sustitución de comandos. ¿Qué significa esto?
- e. Realizar modificaciones al script anteriormente creado de manera de poder mostrar distintos resultados (cuál es su directorio personal, el contenido de un directorio en particular, el espacio libre en disco, etc.). Pida que se introduzcan por teclado (entrada estándar) otros datos.

4. Parametrización: ¿Cómo se acceden a los parámetros enviados al script al momento de su invocación? ¿Qué información contienen las variables `$#`, `$*`, `$?` y `$HOME` dentro de un script?
5. ¿Cuál es la funcionalidad de comando `exit`? ¿Qué valores recibe como parámetro y cuál es su significado?
6. El comando `expr` permite la evaluación de expresiones. Su sintaxis es:
 `expr arg1 op arg2,`
donde `arg1` y `arg2` representan argumentos y `op` la operación de la expresión. Investigar que tipo de operaciones se pueden utilizar.
7. El comando `test expresión` permite evaluar expresiones y generar un valor de retorno, `true` o `false`. Este comando puede ser reemplazado por el uso de corchetes de la siguiente manera `[ expresión ]`. Investigar qué tipo de expresiones pueden ser usadas con el comando `test`. Tenga en cuenta operaciones para: evaluación de archivos, evaluación de cadenas de caracteres y evaluaciones numéricas.
8. Estructuras de control. Investigue la sintaxis de las siguientes estructuras de control incluidas en shell scripting:
 - `if`
 - `case`
 - `while`
 - `for`
 - `select`
9. ¿Qué acciones realizan las sentencias `break` y `continue` dentro de un bucle? ¿Qué parámetros reciben?
10. ¿Qué tipo de variables existen? ¿Es shell script fuertemente tipado? ¿Se pueden definir arreglos? ¿Cómo?
11. ¿Pueden definirse funciones dentro de un script? ¿Cómo? ¿Cómo se maneja el pasaje de parámetros de una función a la otra?
12. Evaluación de expresiones.
 - a. Realizar un script que le solicite al usuario 2 números, los lea de la entrada Standard e imprima la multiplicación, suma, resta y cual es el mayor de los números leídos.
 - b. Modificar el script creado en el inciso anterior para que los números sean

recibidos como parámetros. El script debe controlar que los dos parámetros sean enviados.

- c. Realizar una calculadora que ejecute las 4 operaciones básicas: +, -, *, %. Esta calculadora debe funcionar recibiendo la operación y los números como parámetros

13. Uso de las estructuras de control:

- a. Realizar un script que visualice por pantalla los números del 1 al 100 así como sus cuadrados.
- b. Crear un script que muestre 3 opciones al usuario: Listar, DondeEstoy y QuienEsta. Según la opción elegida se le debe mostrar:
 - Listar: lista el contenido del directorio actual.
 - DondeEstoy: muestra la ruta del directorio donde me encuentro ubicado.
 - QuienEsta: muestra los usuarios conectados al sistema.
- c. Crear un script que reciba como parámetro el nombre de un archivo e informe si el mismo existe o no, y en caso afirmativo indique si es un directorio o un archivo. En caso de que no exista el archivo/directorio cree un directorio con el nombre recibido como parámetro.

14. Renombrando Archivos: haga un script que renombre solo archivos de un directorio pasado como parámetro, agregándole una CADENA, contemplando las opciones:

- "-a CADENA": renombra el fichero concatenando CADENA al final del nombre del archivo
- "-b CADENA": renombra el fichero concatenando CADENA al comienzo del nombre del archivo

Ejemplos:

Si tengo los siguientes archivos: /tmp/a /tmp/b , al ejecutar: **./renombrar /tmp/ -a EJ** obtendré como resultado: /tmp/aEJ /tmp/bEJ. Y si ejecuto: **./renombrar /tmp/ -b EJ** el resultado será: /tmp/EJa /tmp/EJb

15. El comando *cut* nos permite procesar las líneas de la entrada que reciba (archivo, entrada estándar, resultado de otro comando, etc) y cortar columnas o campos, siendo posible indicar cuál es el delimitador de las mismas. Investigue los parámetros que puede recibir este comando y cite ejemplos de uso.
16. Realizar un script que reciba como parámetro una extensión y haga un reporte con 2 columnas, el nombre de usuario y la cantidad de archivos que posee con esa extensión. Se debe guardar el resultado en un archivo llamado *reporte.txt*
17. Escribir un script que al ejecutarse imprima en pantalla los nombres de los

archivos que se encuentran en el directorio actual, intercambiando minúsculas por mayúsculas, además de eliminar la letra a (mayúscula o minúscula).
Por ejemplo, si en el directorio actual están los siguientes archivos:

- lsO
- pepE
- Maria

y ejecutó: `./ejercicio17` , se obtendrá como resultado:

- iSo
- PEPe
- mRI

Ayuda: Investigar el comando `tr`

18. Crear un script que verifique cada 10 segundos si un usuario se ha logueado en el sistema (el nombre del usuario será pasado por parámetro). Cuando el usuario finalmente se loguee, el programa deberá mostrar el mensaje "Usuario XXX logueado en el sistema" y salir.
19. Escribir un Programa de "Menu de Comandos Amigable con el Usuario" llamado menú, el cual, al ser invocado, mostrará un menú con la selección para cada uno de los scripts creados en esta práctica. Las instrucciones de cómo proceder deben mostrarse junto con el menú. El menú deberá iniciarse y permanecer activo hasta que se seleccione Salir. Por ejemplo:

```
MENU DE COMANDOS
03. Ejercicio 3
12. Evaluar Expresiones
13. Probar estructuras de control
...
Ingrese la opción a ejecutar: 03
```

20. Realice un script que simule el comportamiento de una estructura de PILA e implemente las siguientes funciones aplicables sobre una estructura global definida en el script:
 - ➔ push: Recibe un parámetro y lo agrega en la pila
 - ➔ pop: Saca un elemento de la pila
 - ➔ length: Devuelve la longitud de la pila
 - ➔ print: Imprime todos elementos de la pila

Dentro del mismo script y utilizando las funciones implementadas:

1. Agregue 10 elementos a la pila

2. Saque 3 de ellos
3. Imprima la longitud de la pila
4. Luego imprima la totalidad de los elementos que en ella se encuentran.

21. Dada la siguiente declaración al comienzo de un script:

```
num=(10 3 5 7 9 3 5 4)
```

(la cantidad de elementos del arreglo puede variar).

Implemente la función **productoria** dentro de este script, cuya tarea sea multiplicar todos los números que el arreglo contiene.

22. Implemente un script que recorra un arreglo compuesto por números e imprima en pantalla sólo los números pares y que cuente sólo los números impares y los informe en pantalla al finalizar el recorrido.

23. Dada la definición de 2 vectores del mismo tamaño y cuyas longitudes no se conocen.

```
vector1=( 1 .. N)
```

```
vector2=( 1.. N)
```

Por ejemplo:

```
vector1=( 1 80 65 35 2 ) y vector2=( 5 98 3 41 8 ).
```

Complete este script de manera tal de implementar la suma elemento a elemento entre ambos vectores y que la misma sea impresa en pantalla de la siguiente manera:

- La suma de los elementos de la posición 0 de los vectores es 6
- La suma de los elementos de la posición 1 de los vectores es 178 ...
- La suma de los elementos de la posición 4 de los vectores es 10

24. Realice un script que agregue en un arreglo todos los nombres de los usuarios del sistema pertenecientes al grupo "users". Adicionalmente el script puede recibir como parámetro:

- "-b n": Retorna el elemento de la posición n del arreglo si el mismo existe. Caso contrario, un mensaje de error.
- "-l": Devuelve la longitud del arreglo
- "-i": Imprime todos los elementos del arreglo en pantalla

25. Escriba un script que reciba una cantidad desconocida de parámetros al momento de su invocación (debe validar que al menos se reciba uno). Cada parámetro representa la ruta absoluta de un archivo o directorio en el sistema. El script deberá iterar por todos los parámetros recibidos, y solo para aquellos parámetros que se encuentren en posiciones impares (el primero, el tercero, el

verificar si el archivo o directorio existen en el sistema, imprimiendo en pantalla que tipo de objeto es (archivo o directorio). Además, deberá informar la cantidad de archivos o directorios inexistentes en el sistema.

26. Realice un script que implemente a través de la utilización de funciones las operaciones básicas sobre arreglos:
- inicializar: Crea un arreglo llamado array vacío
 - agregar_elem <parametro1>: Agrega al final del arreglo el parámetro recibido
 - eliminar_elem <parametro1>: Elimina del arreglo el elemento que se encuentra en la posición recibida como parámetro. Debe validar que se reciba una posición válida
 - longitud: Imprime la longitud del arreglo en pantalla
 - imprimir: Imprime todos los elementos del arreglo en pantalla
 - inicializar_Con_Valores <parametro1><parametro2>: Crea un arreglo con longitud <parametro1>y en todas las posiciones asigna el valor <parametro2>
27. Realice un script que reciba como parámetro el nombre de un directorio. Deberá validar que el mismo exista y de no existir causar la terminación del script con código de error 4. Si el directorio existe deberá contar por separado la cantidad de archivos que en él se encuentran para los cuales el usuario que ejecuta el script tiene permiso de lectura y escritura, e informar dichos valores en pantalla. En caso de encontrar subdirectorios, no deberán procesarse, y tampoco deberán ser tenidos en cuenta para la suma a informar.
28. Implemente un script que agregue a un arreglo todos los archivos del directorio /home cuya terminación sea .doc. Adicionalmente, implemente las siguientes funciones que le permitan acceder a la estructura creada:
- verArchivo <nombre_de_archivo>: Imprime el archivo en pantalla si el mismo se encuentra en el arreglo. Caso contrario imprime el mensaje de error "Archivo no encontrado" y devuelve como valor de retorno 5
 - cantidadArchivos: Imprime la cantidad de archivos del /home con terminación .doc
 - borrarArchivo <nombre_de_archivo>: Consulta al usuario si quiere eliminar el archivo lógicamente. Si el usuario responde Si, elimina el elemento solo del arreglo. Si el usuario responde No, elimina el archivo del arreglo y también del FileSystem. Debe validar que el archivo exista en el arreglo. En caso de no existir, imprime el mensaje de error "Archivo no encontrado" y devuelve como valor de retorno 10
29. Realice un script que mueva todos los programas del directorio actual (archivos

ejecutables) hacia el subdirectorio "bin" del directorio HOME del usuario actualmente logueado. El script debe imprimir en pantalla los nombres de los que mueve, e indicar cuántos ha movido, o que no ha movido ninguno. Si el directorio "bin" no existe, deberá ser creado.

30. Implemente la estructura de datos Set (Conjunto de valores) en Bash. Un conjunto se define como una colección de valores únicos, es decir que solo almacena una vez cada valor, aún cuando se intente agregar el mismo valor más de una vez. La implementación debe soportar las siguientes operaciones mediante funciones:

- **initialize** - inicializa el set vacío.
- **initialize_with** - inicializa el set con un conjunto de valores que recibe como argumento (debe validar que se reciba al menos uno).
- **add** - Agrega un valor al conjunto, el cual recibe como argumento. No debe agregar elementos repetidos. El resultado de la operación será un éxito solo si el valor puede ser agregado al conjunto.
- **remove** - Elimina uno o más valores del conjunto, los cuales recibe como argumentos. Si la operación elimina al menos un valor, se considera un éxito.
- **contains** - Chequea si el conjunto contiene un valor recibido como argumento. El resultado será éxito si el valor está en el conjunto.
- **print** - Imprime los elementos del conjunto, de a uno por línea.
- **print_sorted** - Imprime los elementos del conjunto, de a uno por línea y ordenados alfabéticamente. *Tip: Investigar cómo combinar el comando sort con la función print.*

En un script separado, incorporar y utilizar las funciones implementadas para desarrollar un juego de bingo. El bingo deberá generar números aleatorios dentro de un rango entre 0 y un valor máximo que puede especificarse mediante un argumento del script, de manera opcional. El valor máximo no puede ser 0 ni superior a 32767, y en caso de no especificarse se tomará como valor por defecto 99. En cada ronda se generará un nuevo número que ya no haya sido utilizado y se lo cantará, imprimiendo en la salida estándar. Luego de esto, se esperará entrada del usuario para saber si se debe cantar "BINGO" para finalizar la partida o se debe cantar un nuevo número. Al finalizar, el script deberá imprimir en orden los números se cantaron hasta que se produjo el bingo.

Tip: Investigar la variable de entorno \$RANDOM de Bash para obtener valores aleatorios.

31. Realice un script que reciba como argumento una lista de posibles nombres de usuarios del sistema y, para cada uno de los que efectivamente existan en el sistema y posean un directorio personal configurado que sea válido, realice las modificaciones necesarias en su directorio personal para que tenga un subdirectorio llamado "directorio_iso" con la siguiente estructura:

```
directorio_iso
├── 2025
│   ├── 01
│   │   └── archivo.txt
│   ├── 02
│   │   └── archivo.txt
│   ├── 03
│   │   └── archivo.txt
│   ├── 04
│   │   └── archivo.txt
│   ├── 05
│   │   └── archivo.txt
│   ├── 06
│   │   └── archivo.txt
│   ├── 07
│   │   └── archivo.txt
│   ├── 08
│   │   └── archivo.txt
│   ├── 09
│   │   └── archivo.txt
│   ├── 10
│   │   └── archivo.txt
│   ├── 11
│   │   └── archivo.txt
│   └── 12
│       └── archivo.txt
└── 2026
    ├── 01
    │   └── archivo.txt
    ├── 02
    │   └── archivo.txt
    ├── 03
    │   └── archivo.txt
    ├── 04
    │   └── archivo.txt
    ├── 05
    │   └── archivo.txt
    ├── 06
    │   └── archivo.txt
    ├── 07
    │   └── archivo.txt
    ├── 08
    │   └── archivo.txt
    └── 09
```

```
|      └─ archivo.txt
├── 10
|      └─ archivo.txt
├── 11
|      └─ archivo.txt
└── 12
      └─ archivo.txt
```

Para resolver la creación de los directorios y archivos utilice la funcionalidad "Brace Expansion" brindada por bash:

https://www.gnu.org/software/bash/manual/html_node/Brace-Expansion.html

Conceptos de Sistemas Operativos (IC)

Introducción a los Sistemas Operativos (LI, LS, APU, ATIC)

Trabajo Práctico N° 4

Objetivo

Continuar los temas relacionados a la administración de memoria, introduciendo paginación por demanda y su utilización para implementar memoria virtual, analizando el impacto en la ejecución de los procesos.

Analizar el componente de Entrada/Salida de un Kernel, su organización, objetivos y funcionalidades en la administración de los dispositivos. Estudiar la implementación de Sistemas de Archivos. Estudiar el impacto de la utilización de Cache de Disco.

Temas incluidos

Paginación por demanda. Algoritmos de reemplazos. Memoria Virtual. Entrada-Salida. Dispositivos. Administración de Discos. Sistemas de Archivos. Buffer Cache.

- Memoria Virtual
 - Describa qué beneficios introduce este esquema de administración de la memoria.
 - ¿En que se debe apoyar el Kernel para su implementación?
 - Al implementar esta técnica utilizando paginación por demanda, las tablas de páginas de un proceso deben contar con información adicional además del marco donde se encuentra la página. ¿Cuál es esta información? ¿Por qué es necesaria?
- Fallos de Página (Page Faults):
 - ¿Cuándo se producen?
 - ¿Quién es responsable de detectar un fallo de página?
 - Describa las acciones que emprende el Kernel cuando se produce un fallo de página.
- Suponga que la tabla de páginas para un proceso que se está ejecutando es la que se muestra a continuación:

Página	Bit V	Bit R	Bit M	Marco
0	1	1	0	4
1	1	1	1	7
2	0	0	0	-
3	1	0	0	2
4	0	0	0	-
5	1	0	1	0

Asumiendo que:

- ☐ El tamaño de la página es de 512 bytes
- ☐ Cada dirección de memoria referencia 1 byte
- ☐ Los marcos se encuentran contiguos y en orden en memoria (0, 1, 2..) a partir de la dirección física 0.

¿Qué dirección física, si existe, correspondería a cada una de las siguientes direcciones virtuales? (No gestionar ningún fallo de página en caso de producirse)

- | | |
|---------|---------|
| a. 1052 | c. 5499 |
| b. 2221 | d. 3101 |

4. Analice cómo impacta el tamaño de una página (pequeña o grande) en paginación por demanda.
5. Asignación de marcos a un proceso (Conjunto de trabajo o Working Set). Con la memoria virtual paginada, no se requiere que todas las páginas de un proceso se encuentren en memoria. El Kernel debe controlar cuantas páginas de un proceso puede tener en la memoria principal. Existen 2 políticas que se pueden utilizar:
 - ☐ Asignación Fija
 - ☐ Asignación Dinámica.
 - a. Describa cómo trabajan estas 2 políticas.
 - b. Dada la siguiente tabla de procesos y las páginas que ellos ocupan, y teniéndose 40 marcos en la memoria principal, cuántos marcos le corresponderá a cada proceso si se usa la técnica de Asignación Fija:
 - Reparto Equitativo
 - Reparto Proporcional

Proceso	Total de Páginas Requeridas
1	15
2	20
3	20
4	8

- c. ¿Cuál de los 2 repartos usados en b) resultó más eficiente? Justifique
6. Reemplazo de páginas (selección de una víctima). ¿Qué sucede cuando todos los marcos en la memoria principal están usados por las páginas de los procesos y se produce un fallo de página? El Kernel debe seleccionar una de las páginas que se encuentra en memoria principal (en un marco) como víctima, la que será reemplazada por la nueva página que produjo el fallo.

Considere los siguientes algoritmos de selección de víctimas básicos:

- LRU
- FIFO
- ☐ OPT (Óptimo)

- ☐ Segunda Chance
 - a. Clasifique estos algoritmos de malo a bueno de acuerdo a la tasa de fallos de página que se obtienen al utilizarlos.
 - b. Analice su funcionamiento. ¿Cómo se podrían implementar?
 - c. Sabemos que la página a ser reemplazada puede estar modificada. ¿Cómo detecta el Kernel esta situación? ¿Qué acciones adicionales deben realizarse cuando se encuentra ante esta situación?
- 7. Alcance del reemplazo. Al momento de tener que seleccionar una página víctima, el Kernel puede optar por 2 políticas a utilizar:
 - ☐ Reemplazo local
 - ☐ Reemplazo global
 - a. Describa cómo trabajan estas 2 políticas.
 - b. ¿Es posible utilizar la política de "Asignación Fija" de marcos junto con la política de "Reemplazo Global"? Justifique.
- 8. Considere la siguiente secuencia de referencias de páginas:
1, 2, 15, 4, 6, 2, 1, 5, 6, 10, 4, 6, 7, 9, 1, 6, 12, 11, 12, 2, 3, 1, 8, 1, 13, 14, 15, 3, 8
 - a. Si se disponen 5 marcos. ¿Cuántos fallos de página se producirán si se utilizan las siguientes técnicas de selección de víctima? (Considere una política de Asignación Dinámica y Reemplazo Global)
 - i. Segunda Chance
 - ii. FIFO
 - iii. LRU
 - iv. OPT
 - b. Suponiendo que cada atención de un fallo de página requiere de 0,1 seg. Calcular el tiempo consumido por atención a los fallos de páginas para los algoritmos de a).
- 9. Sean los procesos A, B y C tales que necesitan para su ejecución las siguientes páginas:
 - ☐ **A:** 1, 3, 1, 2, 4, 1, 5, 1, 4, 7, 9, 4
 - ☐ **B:** 2, 4, 6, 2, 4, 1, 8, 3, 1, 8
 - ☐ **C:** 1, 2, 4, 8, 6, 1, 4, 1

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguientes secuencia:

- | | |
|-------------------------------|--------------------------------|
| 1. B demanda 2 páginas | 8. C demanda 4 páginas |
| 2. A demanda 3 páginas | 9. A demanda 3 páginas |
| 3. C demanda 2 páginas | 10. B demanda 3 páginas |
| 4. B demanda 3 páginas | 11. C termina |
| 5. A demanda 3 páginas | 12. A demanda 3 páginas |
| 6. C demanda 2 páginas | 13. B termina |
| 7. B demanda 2 páginas | 14. A termina |

- a. Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctimas:
- LRU
 - Segunda Chance
- b. Considerando una política de Asignación Fija con reparto equitativo y Reemplazo Local y disponiéndose de 9 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctimas:
- LRU
 - Segunda Chance

10. Sean los procesos A, B y C tales que necesitan para su ejecución las siguientes páginas:

- ☐ **A:** 1, 2, 1, 7, 2, 7, 3, 2
- ☐ **B:** 1, 2, 5, 2, 1, 4, 5
- ☐ **C:** 1, 3, 5, 1, 4, 2, 3

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguiente manera:

- | | |
|-----------------------------------|--------------------------------|
| 1. C demanda 1 página | 12. B demanda 2 páginas |
| 2. A demanda 2 páginas | 13. A demanda 1 página |
| 3. C demanda 1 página | 14. B demanda 2 páginas |
| 4. B demanda 1 página | 15. C demanda 2 páginas |
| 5. A demanda 1 página | 16. C demanda 1 página |
| 6. C modifica la página 1 | 17. A demanda 1 página |
| 7. B demanda 2 páginas | 18. B termina |
| 8. A demanda 1 página | 19. A demanda 2 páginas |
| 9. C demanda 1 página | 20. C demanda 1 página |
| 10. B modifica la página 2 | 21. A termina |
| 11. A modifica la página 2 | 22. C termina |

Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos, debiéndose guardar 1 marco para la gestión de descarga asincrónica de páginas modificadas ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctima:

- Segunda Chance
- FIFO
- LRU

11. Hiperpaginación (Trashing)

- ¿Qué es?
- ¿Cuáles pueden ser los motivos que la causan?
- ¿Cómo la detecta el SO?
- Una vez que lo detecta, ¿qué acciones puede tomar el Kernel para eliminar este problema?

12. Analice y compare las técnicas de PFF (Page Fault Frequency) y del Working Set. ¿Como las

mismas permiten determinar la existencia de trashing?

13. Considere un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda que utiliza un dispositivo de paginación, algoritmo de reemplazo global LRU y una política de asignación que reparte marcos equitativamente entre los procesos. El nivel de multiprogramación es actualmente, de 4.

Ante las siguientes mediciones:

- a) Uso de CPU del 13%, uso del dispositivo de paginación del 97%.
- b) Uso de CPU del 87%, uso del dispositivo de paginación del 3%.
- c) Uso de CPU del 13%, uso del dispositivo de paginación del 3%.

Analizar para cada caso:

- ☐ ¿Qué sucede?
- ☐ ¿Puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU?
- ☐ ¿La paginación está siendo útil para mejorar el rendimiento del sistema?

14. Considere un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda. Considere las siguientes medidas de utilización:

- ☐ Utilización del procesador: 20%
- ☐ Utilización del dispositivo de paginación: 97,7%
- ☐ Utilización de otros dispositivos de E/S: 5%

Cuales de las siguientes acciones pueden mejorar la utilización del procesador:

- a) Instalar un procesador más rápido
- b) Instalar un dispositivo de paginación mayor
- c) Incrementar el grado de multiprogramación
- d) Instalar mas memoria principal
- e) Decrementar el quantum para cada proceso

15. La siguiente fórmula describe el tiempo de acceso efectivo a la memoria al utilizar paginación para la implementación de la memoria virtual:

$$TAE = At + [(1 - p) * Am] + [p * (Tf + Am)]$$

Donde:

- TAE = tiempo de acceso efectivo
- p = tasa de fallo de página ($0 \leq p \leq 1$)
- Am = tiempo de acceso a la memoria real
- Tf = tiempo de atención de una fallo de pagina
- At = tiempo de acceso a la tabla de páginas. Es igual al tiempo de acceso a la memoria (Am) si la entrada de la tabla de páginas no se encuentra en la TLB.

Suponga que tenemos una memoria virtual paginada, con tabla de páginas de 1 nivel, y donde la tabla de páginas se encuentra completamente en la memoria.

Servir una falla de página tarda 300 nanosegundos si hay disponible un marco vacío o si la página reemplazada no se ha modificado, y 500 nanosegundos si se ha modificado. El tiempo de acceso a memoria es de 20 nanosegundos y el de acceso a la TLB es de 1 nanosegundo

- a. Si suponemos una tasa de fallos de página de 0,3 y que siempre contamos con un marco libre para atender el fallo ¿Cuál será el TAE si el 50% de las veces la entrada de

la tabla de páginas se encuentra en la TLB (hit)?

- Si suponemos una tasa de fallos de página de 0,3; que el 70% de las ocasiones la página a reemplazar se encuentra modificada. ¿Cuál será el TAE si el 60% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit)?
- Si suponemos que el 60% de las veces la página a reemplazar está modificada, el 100% de las veces la entrada de la tabla de páginas requerida se encuentra en la TLB (hit) y se espera un TAE menor a 200 nanosegundos.
- ¿Cuál es la máxima tasa aceptable de fallas de página?

16. Considere el siguiente programa:

```
#define Size 64
int A[Size; Size], B[Size; Size], C[Size;
Size]; int register i, j;
for (j = 0; j < Size; j++)
    for (i = 0; i < Size; i++)
        C[i; j] = A[i; j] + B[i; j];
```

Si asumimos que el programa se ejecuta en un sistema que utiliza paginación por demanda para administrar la memoria, donde cada página es de 1Kb. Cada número entero (int) ocupa 4 bytes. Es claro que cada matriz requiere de 16 páginas para almacenarse. Por ejemplo: A[0,0]..A[0,63], A[1,0]..A[1,63], A[2,0]..A[2,63] y A[3,0]..A[3,63] se almacenará en la primera página.

Asumamos que el sistema utiliza un working set de 4 marcos para este proceso. Uno de los 4 marcos es utilizado por el programa y los otros 3 se utilizan para datos (las matrices). También consideramos que para los índices "i" y "j" se utilizan 2 registros, por lo que no es necesario el acceso a la memoria para estas 2 variables.

- Analizar cuántos fallos de páginas ocurren al ejecutar el programa (considere las veces que se ejecuta $C[i,j] = A[i,j] + B[i,j]$)
- Puede ser modificado el programa para minimizar el número de fallos de páginas. En caso de ser posible indicar la cantidad de fallos de páginas que ocurren.

17. Considere las siguientes secuencias de referencias a páginas de los procesos A y B, donde se muestra en instante de tiempo en el que ocurrió cada una (1 a 78):

Proceso A																																							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
1	2	1	3	4	3	3	2	1	1	3	4	5	6	1	2	1	3	3	4	5	6	6	6	5	4	3	1	1	2	3	4	5	4	3	2	1	1	1	
4	4	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	2	1	2	1	1	2	3	4	5	6	7	8	6	5	3	1	2	3	4	5	6	5	1	1	2	3	4	5	4	2	1	1	2	3	4	5	1	1	

Proceso B																																							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
1	2	3	3	3	4	4	5	1	1	1	2	3	4	4	4	4	1	1	2	3	6	5	6	5	4	6	1	1	1	1	4	5	1	3	2	1	1	2	
4	4	4	4	4	4	4	4	4	4	50	5	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	2	3	3	4	4	4	3	4	5	4	6	1	1	1	2	3	3	3	4	5	5	5	1	6	2	3	1	2	3	1	1	1	2	2	2	2	3	3	

- a. Considerando una ventana $\Delta=5$, indique cuál sería el conjunto de trabajo de los procesos A y B en el instante 24 ($WS_A(24)$ y $WS_B(24)$)
- b. Considerando una ventana $\Delta=5$, indique cuál sería el conjunto de trabajo de los procesos A y B en el instante 60 ($WS_A(60)$ y $WS_B(60)$)
- c. Para el los WS obtenidos en el inciso a), si contamos con 8 frames en el sistema ¿Se puede indicar que estamos ante una situación de trashing? ¿Y si contáramos con 6 frames?
- d. Considerando únicamente el proceso A, y suponiendo que al mismo se le asignaron inicialmente 4 marcos, donde el de reemplazo de páginas es realizado considerando el algoritmo FIFO. ¿Cuál será la tasa de fallos en el instante 38 de páginas suponiendo que la misma se calcula contando los fallos de páginas que ocurrieron en las últimas 10 unidades de tiempo?
- e. Para el valor obtenido en el inciso d), si suponemos que el S.O. utiliza como límites superior e inferior de tasa de fallos de páginas los valores 2 y 5 respectivamente ¿Qué acción podría tomar el Kernel respecto a la cantidad de marcos asignados al proceso?

Administración de E/S

18. Dispositivos

- a. Los dispositivos, según la forma de transferir los datos, se pueden clasificar en 2 tipos:
 - i. Orientados a bloques
 - ii. Orientados a flujos
- b. Describa las diferencias entre ambos tipos.
- c. Cite ejemplos de dispositivos de ambos tipos.
- d. Enuncie las diferencias que existen entre los dispositivos de E/S y que el Kernel debe considerar.

19. Técnicas de E/S. Describa cómo trabajan las siguientes técnicas de E/S

- a. E/S programada
- b. E/S dirigida por interrupciones
- c. DMA (Acceso Directo a Memoria)

20. La técnica de E/S programa puede trabajar de dos formas:

- a. E/S mapeada
- b. E/S aislada

Indique cómo trabajan estas 2 técnicas.

21. Enuncie las metas que debe perseguir un SO para la administración de la entrada/salida.

22. Drivers

- a. ¿Qué son?
- b. ¿Qué funciones mínimas deben proveer?
- c. ¿Quién determina cuáles deben ser estas funciones?

23. Realice un gráfico que marque la relación entre el Subsistema de E/S, los drivers, los controladores de dispositivos y los dispositivos.

24. Describa mediante un ejemplo los pasos mínimos que se suceden desde que un proceso genera un requerimiento de E/S hasta que el mismo llega al dispositivo.
25. Describa mediante un ejemplo los pasos mínimos que se suceden desde que un proceso genera un requerimiento de E/S hasta que el mismo llega al dispositivo.
26. Enuncie qué servicios provee el Kernel para la administración de E/S.

Administración de Discos

27. Describa en forma sintética, cómo es la organización física de un disco, puede utilizar gráficos para mayor claridad.
28. La velocidad promedio para la obtención de datos de un disco está dada por la suma de los siguientes tiempos:
- *Seek Time*
 - *Latency Time*
 - *Transfer Time*

De una definición para cada uno.

29. Suponga un disco con las siguientes características:
- 7 platos con 2 caras utilizables cada uno.
 - 1100 cilindros, con 300 sectores por pista, donde cada sector es 512 bytes.
 - Seek Time de 10 ms
 - 9000 RPM .
 - Velocidad de Transferencia de 10 MiB/s (Mebibytes por segundos).
- a. Calcule la capacidad total del disco.
- b. ¿Cuántos sectores de disco se requerirían si se quiere almacenar 3 MiB(MebiBytes) de datos?
- c. Calcule el tiempo de latencia.
30. Originalmente, los sectores de un disco se identificaban por la terna de valores **(C,H,S)** esto es Cylinder-head-sector (Cilindro, Cabeza , Sector), con **C** desde 0 hasta la cantidad de cilindros menos 1, **H** desde 0 hasta la cantidad de cabezas lecto/grabadoras menos 1 y **S** desde 1 hasta la cantidad de sectores por pista.

En los '90 los controladores de disco reemplazaron la identificación de los sectores por un modelo numérico denominado LBA (Logical Block Addressing). La numeración comenzaba en 0 y alcanzaba la cantidad total de sectores menos 1. Cada sector en formato (C,H,S) se puede traducir al LBA a partir de la siguiente fórmula:

$$A = (C * N_{heads} + H) * N_{sectors} + (S - 1)$$

Donde:

- ➔ A es el valor LBA a obtener
- ➔ N_{heads} es la cantidad de cabezas lecto/grabadoras
- ➔ $N_{sectors}$ es la cantidad de sectores por pista

También se puede obtener el valor de la terna (C,H,S) a partir de un valor LBA con las

siguientes fórmulas:

$C = LBA \text{ div } (N_{heads} * N_{sectors})$	$H = \text{Resto de } (LBA \text{ mod } (N_{heads} * N_{sectors})) / N_{sectors}$	$S = (LBA \text{ mod } (N_{heads} * N_{sectors})) \text{ mod } N_{sectors} + 1$
--	---	---

- a. Si se tiene un disco con 1020 cilindros, 16 cabezas y 63 sectores por pista, calcule la cantidad total de sectores que tendrá el disco y determine el valor LBA para las siguientes ternas (C,H,S):

i. (3, 2, 1)

iii. (1019, 15, 63)

ii. (0, 0, 1)

iv. (500, 8, 44)

- b. Determine el valor de la terna (C,H,S) para los siguientes valores LBA:

i. 1

iii. 14890

ii. 100

iv. 45687

31. El *Seek Time* es el parámetro que posee mayor influencia en el tiempo real necesario para transferir datos desde o hacia un disco. Es importante que el Kernel planifique los diferentes requerimientos al disco para minimizar el movimiento de la cabeza lecto-grabadora.

Analicemos las diferentes políticas de planificación de requerimientos a disco con un ejemplo: Supongamos un *Head* con movimiento en 200 *tracks* (numerados de 0 a 199), que está en el track 83 atendiendo un requerimiento y anteriormente atendió un requerimiento en el track 75.

Si la cola de requerimientos es: {86, 147, 91, 177, 94, 150, 102, 175, 130, 32, 120, 58, 66, 115}. Realice los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

a. FCFS (*First Come, First Served*)

d. Look

b. SSTF (*Shortest Seek Time First*)

e. C-Scan (*Circular Scan*)

c. Scan

f. C-Look (*Circular Look*)

32. ¿Alguno de los algoritmos analizados en el ejercicio anterior pueden causar inanición de requerimientos?

33. Supongamos un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 143 atendiendo un requerimiento y anteriormente atendió un requerimiento en la pista 125.

Si la cola inicial de requerimientos es: {126, 147, 81, 277, 94, 150, 212, 175, 140, 225, 280, 50, 99, 118, 22, 55} y luego de 30 movimientos se incorporan los requerimientos de las pistas {75, 115, 220 y 266}. Realice los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

a. FCFS

d. Look

b. SSTF

e. C-Scan

c. Scan

f. C-Look

34. Supongamos un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 140 atendiendo un requerimiento y anteriormente atendió un

requerimiento en la pista 135.

Si la cola de requerimientos es: {99, 110, 42, 25, 186, 270, 50, 99, 147^{PF}, 81, 257, 94, 133, 212, 175, 130} y después de 30 movimientos se incorporan los requerimientos de las pistas {85, 15^{PF}, 202 y 288} y después de otros 40 movimientos más se incorporan los requerimientos de las pistas {75, 149^{PF}, 285, 201 y 59}. Realice los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

- | | |
|---------|-----------|
| a. FCFS | d. Look |
| b. SSTF | e. C-Scan |
| c. Scan | f. C-Look |

Administración de Archivos

35. Dados las siguientes técnicas de administración de espacio de un archivo:
- Asignación contigua
 - Asignación enlazada
 - Asignación indexada
- a. Describa cómo trabaja cada una.
- b. Cite ventajas y desventajas de cada una.
- c. Indique para cada técnica si la misma puede producir fragmentación interna y/o externa.
36. Gestión de espacio libre. Dados los siguientes métodos de gestión de espacio libre en un disco:
- | | |
|------------------|----------------|
| ➤ Tabla de bits | ➤ Agrupamiento |
| ➤ Lista Enlazada | ➤ Recuento |
- a. Describa cómo trabajan estos métodos.
- b. Cite ventajas y desventajas de cada uno.
37. Datos un disco con las características vistas en el ejercicio 29:
- a. Calcule el tiempo de transferencia real de un archivo de 15 MiB(Mebibytes) almacenado en el disco de manera secuencial (todos sus bloques almacenados de manera consecutiva)
- b. Calcule el tiempo de transferencia real de un archivo de 16 MiB(Mebibytes) almacenado en el disco de manera aleatoria.
38. Unidad de Asignación. Las técnicas vistas pueden referenciar a **sectores** físicos de medio de almacenamiento o **clusters** (conjunto consecutivo de sectores físicos). Cite ventajas y desventajas de la utilización de cada forma de unidad de asignación.
39. Si se tiene un disco con 10 platos con 2 caras utilizables cada uno (20 cabezas), 350 cilindros y con 500 sectores por pista ¿Cuántos cluster pueden haber si se toma un tamaño de 4 sectores? ¿Y si el cluster es de 8 sectores?
40. Asumiendo un sistema de archivos que utiliza como unidad de asignación un cluster de 8 sectores de disco y que el primer cluster de datos (cluster 0) se comienza a almacenar en el sector LBA 120. Determine el los valores LBA de los sectores que corresponden a los

siguientes números de cluster:

- a. 0
- b. 123
- c. 299
- d. 1001

41. Describa la organización del FileSystem de Unix System V. Tenga en cuenta la organización del mismo, estructuras de datos, etc.

42. Gestión de archivos en *UNIX* System V

El sistema de archivos de UNIX utiliza una variante del esquema de asignación indexada para la administración de espacio de los archivos. Cada archivo o directorio está representado por una estructura que mantiene, entre otra información, las direcciones de los bloques que contienen los datos del archivo: el I-NODO.

Cada I-NODO contiene 13 campos para direcciones a los bloques de datos, organizadas de la siguiente manera:

- 10 de direccionamiento directo.
- 1 de direccionamiento indirecto simple.
- 1 de direccionamiento indirecto doble.
- 1 de direccionamiento indirecto triple.

- a. Realice un gráfico que represente la estructura del *I-NODO* y de los bloques de datos. Cada cluster es de 1 Kib(Kibibits).
- b. Si cada dirección para referenciar un bloque es de 32 bits:
 - i. ¿Cuántas referencias (direcciones) a bloque pueden contener un cluster?
 - ii. ¿Cuál sería el tamaño máximo de un archivo?

43. Dado el FileSystem de UNIX System V (visto en clase), indique qué cambios se producen en mismo dada las siguientes situaciones:

- a. Se agrega un nuevo archivo "Archivo1" en el directorio "Directorio1"
- b. Se agrega nuevo contenido el archivo "Archivo1"
- c. Se modifica el nombre del archivo "Archivo1" a "Archivo2"
- d. Se hace un link duro a "Archivo2" en el mismo directorio con nombre "LinkArchivo2".
- e. Se borra un archivo "Archivo3".

44. Se tiene un FileSystem con un modelo indexado con niveles de indirección, donde se mantiene un i-nodo con 10 índices de direccionamiento directo, 1 índice para indirecto simple y 2 para indirecto doble. Considerando además que utilizan 32 bits para referenciar direcciones a bloques/clusters.

Suponga que existen dos siguientes archivos con sus tamaños logicos:

- *ArchivoA*: de 55 KibiBytes
- *ArchivoB*: de 12 KibiBytes
- *ArchivoC*: de 1075 KibiBytes
- a. Si se utiliza como unidad de asignación el sector de disco de 512bytes
 - i. ¿Cuántas direcciones a sectores puede contener un sector de disco?
 - ii. ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?
 - iii. ¿Cuántos sectores se necesitan para almacenar cada uno de los archivos antes enumerados? Indique el tamaño físico que se requirió para

almacenar cada uno.

- iv. Indique la fragmentación interna que se produce para cada uno de los archivos antes enumerados.
 - b. Si se utiliza como unidad de asignación el cluster de disco de 2 KibiBytes
 - i. ¿Cuántas direcciones a clusters puede contener un cluster de disco?
 - ii. ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?
 - iii. ¿Cuántos clusters se necesitan para almacenar cada uno de los archivos antes enumerados? Indique el tamaño físico que se requirió para almacenar cada uno.
 - iv. Indique la fragmentación interna que se produce para cada uno de los archivos antes enumerados.
45. Para los incisos a. y b. el ejercicio anterior, y considerando que el fileSystem se utiliza en un disco con las siguientes características:
- 6 platos, con 2 caras útiles cada uno
 - 2500 pistas por cara y 100 sectores por pista de 512 bytes cada uno.
 - 7200 RPM
 - seek type de 10,5 ms
 - velocidad de transferencia de 146 MiB/seg (Mebibytes por segundo)
- Calcule el tiempo de transferencia de los 3 archivos (ArchivoA, ArchivoB y ArchivoC) del ejercicio previo.
46. Analice las siguientes fórmulas necesarias para localizar un I-NODO en la tabla de i-nodos de un filesystem similar a System V (visto en la teoría)

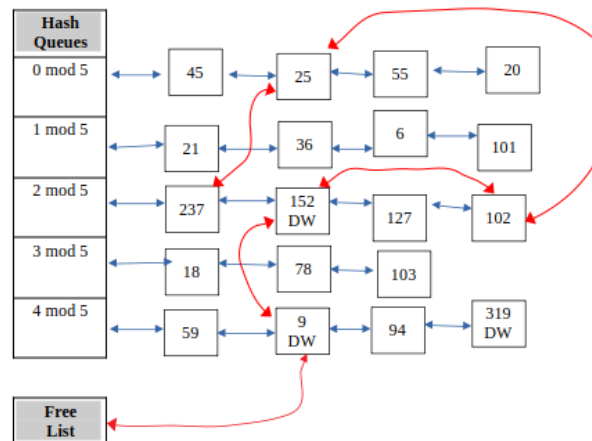
nro bloque = ((nro de inodo - 1)/nro. de inodos por bloque) + bloque de comienzo de la lista de inodos.

Desplazamiento del inodo en el bloque = ((nro de inodo - 1) módulo (número de inodos por bloque)) * medida de inodo del disco.

- a. Según la primera fórmula, asumiendo que en el bloque 2 está en el comienzo de la lista de inodos y que hay 8 inodos por bloque: calcule donde se encuentra el inodo 8 y el 9. ¿Dónde estarían para un bloque de disco con 16 inodos?
- b. De acuerdo a la segunda fórmula, si cada inodo del disco ocupa 64 bytes y hay 8 inodos por bloque de disco, el inodo 8 comienza en el desplazamiento 448 del bloque de disco. ¿Dónde empieza el 6? Si fueran inodos de 128 bytes y 24 inodos por bloque: ¿dónde empezaría el inodo 8?

Caché de Disco

47. Dado el siguiente gráfico que representa el estado de Buffer Cache en Unix System V (Visto en clase):



Responda a los siguientes incisos (considere cambios en los headers, las hash queue, free list, operaciones de E/S, etc):

- ¿Qué sucedería si un proceso P1 requiere el bloque 45?
- ¿Qué sucedería si un proceso P1 solicita el bloque 113?
- ¿Qué sucedería si un proceso P1 solicita el bloque 102?
- ¿Qué sucedería si un proceso P1 solicita el bloque 9?

Conceptos de Sistemas Operativos (IC)

Introducción a los Sistemas Operativos (LI, LS, APU, ATIC)

Trabajo Práctico N° 1

Objetivo

El objetivo de esta práctica es que el estudiante se familiarice con los conceptos básicos del sistema operativo *GNU/Linux*, su instalación, entorno y comandos principales. Manejo de Usuarios, permisos y su sistemas de archivos.

Temas Incluidos

GNU/Linux, instalación y conceptos básicos, permisos, arranque, usuarios. organización interna.

1. Características de *GNU/Linux*:

- Mencione y explique las características más relevantes de *GNU/Linux*.
- Mencione otros sistemas operativos y compárelos con *GNU/Linux* en cuanto a los puntos mencionados en el inciso *a*.
- ¿Qué es **GNU**?
- Indique una breve historia sobre la evolución del proyecto *GNU*.
- Explique qué es la multitarea, e indique si *GNU/Linux* hace uso de ella.
- ¿Qué es **POSIX**?

2. Distribuciones de *GNU/Linux*:

- ¿Qué es una distribución de *GNU/Linux*? Nombre al menos 4 distribuciones de *GNU/Linux* y cite diferencias básicas entre ellas.
- ¿En qué se diferencia una distribución de otra?
- ¿Qué es **Debian**? Acceda al sitio <https://www.debian.org/> e indique cuáles son los objetivos del proyecto y una breve cronología del mismo.

3. Estructura de *GNU/Linux*:

- Nombre cuáles son los 3 componentes fundamentales de *GNU/Linux*.
- Mencione y explique la estructura básica del Sistema Operativo *GNU/Linux*.

4. Kernel:

- ¿Cuáles son sus funciones principales?
- ¿Cuál es la versión actual? ¿Cómo se definía el esquema de versionado del Kernel en versiones anteriores a la 2.4? ¿Qué cambió en el versionado que se impuso a partir de la versión 2.6?
- ¿Es posible tener más de un Kernel de *GNU/Linux* instalado en la misma máquina?
- ¿Dónde se encuentra ubicado dentro del File System?

5. Intérprete de comandos (*Shell*):

- ¿Qué es?
- ¿Cuáles son sus funciones?
- Mencione al menos 3 intérpretes de comandos que posee *GNU/Linux* y compárelos entre ellos.
- ¿Dónde se ubican (*path*) los comandos propios y externos al Shell?
- ¿Por qué considera que el Shell no es parte del Kernel de *GNU/Linux*?
- ¿Es posible definir un intérprete de comandos distinto para cada usuario?
¿Desde dónde se define? ¿Cualquier usuario puede realizar dicha tarea?

6. El sistema de Archivos (*File System*) en Linux:

- ¿Qué es?
- ¿Cuál es la estructura básica de los File System en *GNU/Linux*? Mencione los directorios más importantes e indique qué tipo de información se encuentra en ellos. ¿A qué hace referencia la sigla **FHS**?
- Mencione sistemas de archivos soportados por *GNU/Linux*.
- ¿Es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en *GNU/Linux*?

7. Particiones:

- Definición. Tipos de particiones. Ventajas y Desventajas.
- ¿Cómo se identifican las particiones en *GNU/Linux*? (Considere discos **IDE**, **SCSI** y **SATA**).
- ¿Cuántas particiones son necesarias como mínimo para instalar *GNU/Linux*? Nómbrelas indicando tipo de partición, identificación, tipo de File System y punto de montaje.
- Dar ejemplos de diversos casos de particionamiento dependiendo del tipo de tarea que se deba realizar en su sistema operativo.
- ¿Qué tipo de software para particionar existe? Méncionelos y compare.

8. Arranque (*bootstrap*) de un Sistema Operativo:

- ¿Qué es el **BIOS**? ¿Qué tarea realiza?
- ¿Qué es **UEFI**? ¿Cuál es su función?
- ¿Qué es el **MBR**? ¿Qué es el **MBC**?
- ¿A qué hacen referencia las siglas **GPT**? ¿Qué sustituye? Indique cuál es su formato.
- ¿Cuál es la funcionalidad de un "Gestor de Arranque"? ¿Qué tipos existen? ¿Dónde se instalan? Cite gestores de arranque conocidos.
- ¿Cuáles son los pasos que se suceden desde que se prende una computadora hasta que el Sistema Operativo es cargado (proceso de *bootstrap*)?
- Analice el proceso de arranque en *GNU/Linux* y describa sus principales pasos.
- ¿Cuáles son los pasos que se suceden en el proceso de parada (*shutdown*) de

GNU/Linux?

- i. ¿Es posible tener en una PC *GNU/Linux* y otro Sistema Operativo instalado? Justifique.

9. Archivos y editores:

- a. ¿Cómo se identifican los archivos en *GNU/Linux*?
- b. Investigue el funcionamiento de los editores **vim**, **nano** y **mcedit**, y los comandos **cat**, **more** y **less**.
- c. Cree un archivo llamado "prueba.exe" en su directorio personal usando el **vim**. El mismo debe contener su número de alumno y su nombre.
- d. Investigue el funcionamiento del comando **file**. Pruébelo con diferentes archivos. ¿Qué diferencia nota?
- e. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso de archivos:

- | | |
|------------|------------|
| i. cd | vi. locate |
| ii. mkdir | vii. ls |
| iii. rmdir | viii. pwd |
| iv. ln | ix. cp |
| v. tail | x. mv |
| | xi. find |

10. Indique qué comando es necesario utilizar para realizar cada una de las siguientes acciones. Investigue su funcionamiento y parámetros más importantes:

- a. Cree la carpeta **ISOC SO**
- b. Acceda a la carpeta
- c. Cree dos archivos con los nombres **isocso.txt** e **isocso.csv**
- d. Liste el contenido del directorio actual
- e. Visualizar la ruta donde estoy situado
- f. Busque todos los archivos en los que su nombre contiene la cadena "iso*"
- g. Informar la cantidad de espacio libre en disco
- h. Verifique los usuarios conectados al sistema
- i. Editar a el archivo **isocso.txt** e ingresar Nombre y Apellido
- j. Mostrar en pantalla las últimas líneas de un archivo.

11. Investigue el funcionamiento, parámetros y ubicación (directorio) de los siguientes comandos:

- | | |
|------------|-----------|
| → man | → dmesg |
| → shutdown | → lspci |
| → reboot | → at |
| → halt | → netstat |
| → uname | → head |
| | → tail |

12. Procesos:

- a. ¿Qué es un proceso? ¿A que hacen referencia las siglas PID y PPID? ¿Todos los procesos tienen estos atributos en GNU/Linux? Justifique. Indique qué otros atributos tiene un proceso.
- b. Investigue el funcionamiento, parámetros y ubicación (directorio) de los siguientes comandos relacionados a procesos. En caso de que algún comando no venga por defecto en la distribución que utiliza deberá proceder a instalarlo:

- | | |
|------------|---------------|
| i. top | vii. pkill |
| ii. htop | viii. killall |
| iii. ps | ix. renice |
| iv. pstree | x. xkill |
| v. kill | xi. atop |
| vi. pgrep | xii. nice |

13. Proceso de Arranque *SystemV* (<https://github.com/systeminit/si>):

- a. Enumere los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.
- b. Proceso **INIT**. ¿Quién lo ejecuta? ¿Cuál es su objetivo?
- c. RunLevels. ¿Qué son? ¿Cuál es su objetivo?
- d. ¿A qué hace referencia cada nivel de ejecución según el estándar? ¿Dónde se define qué Runlevel ejecutar al iniciar el sistema operativo? ¿Todas las distribuciones respetan estos estándares?
- e. Archivo `/etc/inittab`. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en él? ¿Cuál es la estructura de la información que en él se almacena?
- f. Suponga que se encuentra en el runlevel `<X>`. Indique qué comando(s) deberá ejecutar para cambiar al runlevel `<Y>`. ¿Este cambio es permanente? ¿Por qué?
- g. Scripts RC. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene se ejecutan scripts, indique cómo determina qué script ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justifique.

14. *SystemD* (<https://github.com/systemd/systemd>):

- a. ¿Qué es *systemd*?
- b. ¿A qué hace referencia el concepto de *Unit* en *SystemD*?
- c. ¿Para qué sirve el comando *systemctl* en *SystemD*?
- d. ¿A qué hace referencia el concepto de *target* en *SystemD*?
- e. Ejecutar el comando *ps tree*. ¿Qué es lo que se puede observar a partir de la ejecución de este comando?

15. Usuarios:

- a. ¿Qué archivos son utilizados en un sistema GNU/Linux para guardar la información de los usuarios?
- b. ¿A qué hacen referencia las siglas *UID* y *GID*? ¿Pueden coexistir *UIDs* iguales en un

sistema GNU/Linux? Justifique.

- c. ¿Qué es el usuario root? ¿Puede existir más de un usuario con este perfil en GNU/Linux? ¿Cuál es la *UID* de root?
- d. Agregue un nuevo usuario llamado *isocso* a su instalación de GNU/Linux, especifique que su home sea creada en */home/isocso*, y hágalo miembro del grupo *informatica* (si no existe, deberá crearlo). Luego, sin iniciar sesión como este usuario cree un archivo en su home personal que le pertenezca. Luego de todo esto, borre el usuario y verifique que no queden registros de él en los archivos de información de los usuarios y grupos.
- e. Investigue la funcionalidad y parámetros de los siguientes comandos:

- | | |
|----------------------|---------------|
| i. useradd y adduser | v. groupadd |
| ii. usermod | vi. who |
| iii. userdel | vii. groupdel |
| iv. su | viii. passwd |

16. FileSystem y permisos:

- a. ¿Cómo son definidos los permisos sobre archivos en un sistema GNU/Linux?
- b. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con los permisos en GNU/Linux:
 - i. chmod
 - ii. chown
 - iii. chgrp
- c. Al utilizar el comando *chmod* generalmente se utiliza una notación octal asociada para definir permisos. ¿Qué significa esto? ¿A qué hace referencia cada valor?
- d. ¿Existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos? Indíquelo y realice las pruebas correspondientes.
- e. Explique los conceptos de “full path name” (path absoluto) y “relative path name” (path relativo). De ejemplos claros de cada uno de ellos.
- f. ¿Con qué comando puede determinar en qué directorio se encuentra actualmente? ¿Existe alguna forma de ingresar a su directorio personal sin necesidad de escribir todo el path completo? ¿Podría utilizar la misma idea para acceder a otros directorios? ¿Cómo? Explique con un ejemplo.
- g. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso del FileSystem:

- | | |
|-----------|-------------------------|
| i. umount | v. mkfs |
| ii. du | vi. fdisk (con cuidado) |
| iii. df | vii. write |
| iv. mount | viii. losetup |
| | ix. stat |

17. Procesos:

- ¿Qué significa que un proceso se está ejecutando en Background? ¿Y en Foreground?
- ¿Cómo puedo hacer para ejecutar un proceso en Background? ¿Como puedo hacer para pasar un proceso de background a foreground y viceversa?
- Pipe (|). ¿Cuál es su finalidad? Cite ejemplos de su utilización.
- Redirección. ¿Qué tipo de redirecciones existen? ¿Cuál es su finalidad? Cite ejemplos de utilización.

18. Otros comandos de Linux (Indique funcionalidad y parámetros):

- ¿A qué hace referencia el concepto de empaquetar archivos en GNU/Linux?
- Seleccione 4 archivos dentro de algún directorio al que tenga permiso y sume el tamaño de cada uno de estos archivos. Cree un archivo empaquetado conteniendo estos 4 archivos y compare los tamaños de los mismos. ¿Qué característica nota?
- ¿Qué acciones debe llevar a cabo para comprimir 4 archivos en uno solo? Indique la secuencia de comandos ejecutados.
- ¿Pueden comprimirse un conjunto de archivos utilizando un único comando?
- Investigue la funcionalidad de los siguientes comandos:

- tar
- grep
- gzip

- zgrep
- wc

19. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando, indique la razón:

```
ls -l > prueba
ps > PRUEBA
chmod 710 prueba
chown root:root PRUEBA
chmod 777 PRUEBA
chmod 700 /etc/passwd
passwd root
rm PRUEBA
man /etc/shadow
find / -name * .conf
usermod root -d /home/ newroot -L
cd / root
rm *
cd / etc
cp * /home -R
shutdown
```

20. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

- a. Terminar el proceso con *PID* 23.
 - b. Terminar el proceso llamado *init* o *systemd*. ¿Qué resultados obtuvo?
 - c. Buscar todos los archivos de usuarios en los que su nombre contiene la cadena ".conf"
 - d. Guardar una lista de procesos en ejecución el archivo **/home/<su nombre de usuario>/procesos**
 - e. Cambiar los permisos del archivo **/home/<su nombre de usuario>/xxxx** a:
 - i. Usuario: Lectura, escritura, ejecución
 - ii. Grupo: Lectura, ejecución
 - iii. Otros: ejecución
 - f. Cambiar los permisos del archivo **/home/<su nombre de usuario>/yyyy** a:
 - i. Usuario: Lectura, escritura.
 - ii. Grupo: Lectura, ejecución
 - iii. Otros: Ninguno
 - g. Borrar todos los archivos del directorio **/tmp**
 - h. Cambiar el propietario del archivo **/opt/isodata** al usuario **isocso**
 - i. Guardar en el archivo **/home/<su nombre de usuario>/donde** el directorio donde me encuentro en este momento, en caso de que el archivo exista no se debe eliminar su contenido anterior.
21. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:
- a. Ingrese al sistema como usuario "root"
 - b. Cree un usuario. Elija como nombre, por convención, la primera letra de su nombre seguida de su apellido. Asígnele una contraseña de acceso.
 - c. ¿Qué archivos fueron modificados luego de crear el usuario y qué directorios se crearon?
 - d. Crear un directorio en */tmp* llamado *miCursada*
 - e. Copiar todos los archivos de */var/log* al directorio antes creado.
 - f. Para el directorio antes creado (y los archivos y subdirectorios contenidos en él) cambiar el propietario y grupo al usuario creado y grupo users.
 - g. Agregue permiso total al dueño, de escritura al grupo y escritura y ejecución a todos los demás usuarios para todos los archivos dentro de un directorio en forma recursiva.
 - h. Acceda a otra terminal para loguearse con el usuario antes creado.
 - i. Una vez logueado con el usuario antes creado, averigüe cuál es el nombre de su terminal.
 - j. Verifique la cantidad de procesos activos que hay en el sistema.
 - k. Verifiqué la cantidad de usuarios conectados al sistema.
 - l. Vuelva a la terminal del usuario root y envíele un mensaje al usuario anteriormente creado enviándole que el sistema va a ser apagado.
 - m. Apague el sistema.
22. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes

acciones:

- n. Cree un directorio cuyo nombre sea su número de legajo e ingrese a él.
- o. Cree un archivo utilizando el editor de textos *vi*, e introduzca su información personal: Nombre, Apellido, Número de alumno y dirección de correo electrónico. El archivo debe llamarse "LEAME".
- p. Cambie los permisos del archivo LEAME, de manera que se puedan ver reflejados los siguientes permisos:
 - Dueño: ningún permiso
 - Grupo: permiso de ejecución
 - Otros: todos los permisos
- q. Vaya al directorio /etc y verifique su contenido. Cree un archivo dentro de su directorio personal cuyo nombre sea **leame** donde el contenido del mismo sea el listado de todos los archivos y directorios contenidos en /etc. ¿Cuál es la razón por la cuál puede crear este archivo si ya existe un archivo llamado "LEAME" en este directorio?
- r. ¿Qué comando utilizaría y de qué manera si tuviera que localizar un archivo dentro del filesystem? ¿Y si tuviera que localizar varios archivos con características similares? Explique el concepto teórico y ejemplifique.
- s. Utilizando los conceptos aprendidos en el punto anterior, busque todos los archivos cuya extensión sea .so y almacene el resultado de esta búsqueda en un archivo dentro del directorio creado en el primer inciso. El archivo deberá llamarse *ejercicioF*.

23. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando indique la razón:

```
01. mkdir iso
02. cd . / iso; ps > f0
03. ls > f1
04. cd /
05. echo $HOME
06. ls -l $> $HOME/ iso/ls
07. cd $HOME; mkdir f2
08. ls -ld f2
09. chmod 341 f2
10. touch dir
11. cd f2
12. cd ~/iso
13. pwd > f3
14. ps | grep 'ps' | wc -l >> ../f2/f3
15. chmod 700 ../f2 ; cd ..
16. find . -name etc/passwd
17. find / -name etc/passwd
18. mkdir ejercicio5
19. . . . . .
20. . . . . .
```

- a. Inicie 2 sesiones utilizando su nombre de usuario y contraseña. En una sesión vaya

siguiendo paso a paso las órdenes que se encuentran escritas en el cuadro superior. En la otra sesión, cree utilizando algún editor de textos un archivo que se llame "explicacion_de_ejercicio" dentro del directorio creado en el ejercicio 22 y, para cada una de los comandos que ejecute en la otra sesión, realice una breve explicación de los resultados obtenidos.

- b. Complete los comandos 19 y 20, de manera tal que realicen la siguiente acción:

19: Copiar el directorio iso y todo su contenido al directorio creado en 24.a

20: Copiar el resto de los archivos y directorios que se crearon en este ejercicio al directorio creado en el ejercicio 24.a

- c. Ejecute las órdenes 19 y 20 y coméntelas en el archivo creado en el inciso a).

24. Cree una estructura desde el directorio /home que incluya varios directorios, subdirectorios y archivos, según el esquema siguiente.



Asuma que "usuario" indica cuál es su nombre de usuario. Además deberá tener en cuenta que dirX hace referencia a directorios y fX hace referencia a archivos. Utilizando la estructura de directorios anteriormente creada, indique qué comandos son necesarios para realizar las siguientes acciones:

- Mueva el archivo "f3" al directorio de trabajo /home/usuario.
- Copie el archivo "f4" en el directorio "dir11".
- Haga lo mismo que en el inciso anterior pero el archivo de destino, se debe llamar "f7".
- Cree el directorio copia dentro del directorio usuario y copie en él, el contenido de "dir1".
- Renombre el archivo "f1" por el nombre *archivo* y vea los permisos del mismo.
- Cambie los permisos del archivo llamado *archivo* de manera de reflejar lo siguiente:
 - Usuario: Permisos de lectura y escritura
 - Grupo: Permisos de ejecución
 - Otros: Todos los permisos
- Renombre los archivos "f3" y "f4" de manera que se llamen "f3.exe" y "f4.exe" respectivamente.
- Utilizando un único comando cambie los permisos de los dos archivos renombrados en el inciso anterior, de manera de reflejar lo siguiente:
 - Usuario: Ningún permiso

- Grupo: Permisos de escritura
- Otros: Permisos de escritura y ejecución

25. Indique qué comando/s es necesario para realizar cada una de las acciones de la siguiente secuencia de pasos (considerando su orden de aparición):
- a. Cree un directorio llamado *logs* en el directorio */tmp*.
 - b. Copie todo el contenido del directorio */var/log* en el directorio creado en el punto anterior.
 - c. Empaquete el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar".
 - d. Empaquete y comprima el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar.gz".
 - e. Copie los archivos creados en c) y d) al directorio de trabajo de su usuario.
 - f. Elimine el directorio creado en a), *logs*.
 - g. Desempaquete los archivos creados en c y d en 2 directorios diferentes.